

DOCUMENTATION TECHNIQUE FINALE

Projet de fin de session

SAVORA

Application mobile de livraison de repas

React Native (Expo · TypeScript) · Node.js / Express · MongoDB

DOCUMENTATION TECHNIQUE

Enseignant : Safouen Bani

Équipe : Global SoftCorporation

ARCHANGE GUIMDO · JORDAN DONGMEZA · YAN SAH · WILFRED GALIMAR

Version du document : v3.1

Date : 2 août 2026

Table des matières

1. Objectifs du projet

But principal

Permettre à un utilisateur de commander un repas auprès d'un restaurant local, de personnaliser sa commande, de payer en ligne de façon sécurisée, de suivre la livraison en temps réel et d'évaluer son expérience après réception.

Par rapport au document de conception du Sprint 1, l'objectif s'est élargi : le produit livré ne se contente pas de simuler le parcours du client, il fait intervenir les trois autres acteurs — restaurant, livreur et administration — qui font réellement avancer une commande.

Utilisateurs visés

Rôle	Ce qu'il fait dans l'application	Statut au Sprint 1	Statut final
Client	Commande, personnalise, paie, suit, discute, note	Prévu	Livré
Restaurant	Reçoit les commandes, fait évoluer leur statut, gère son menu	Minimal	Livré
Livreur	Accepte une course et la mène jusqu'à la livraison	Hors périmètre	Livré
Administrateur	Supervise l'activité, gère comptes, restaurants et menus	Hors périmètre	Livré

Contraintes techniques

- Frontend mobile : Expo SDK 54, React Native 0.81, TypeScript, expo-router.
- Backend : Node.js 20+, Express 4 — API REST.
- Base de données : MongoDB 6+ via Mongoose 8.
- Temps réel : Socket.IO 4, authentifié par le même jeton JWT que l'API.
- Paiement : Stripe en mode test, avec repli sur une simulation locale.
- Notifications : expo-notifications et service Expo Push.
- Sécurité : JWT (24 h) et bcrypt (12 tours).
- Hébergement : Render ou Railway, base MongoDB Atlas.
- Compatibilité : Android, iOS et web depuis une seule base de code.

2. Périmètre livré

Inclus

- Inscription, connexion, gestion du profil, réinitialisation du mot de passe.
- Session conservée entre deux lancements de l'application.
- Menu interactif : catalogue, recherche, fiche restaurant, personnalisation des plats.
- Panier multi-lignes et passage de commande.
- Paiement par carte (Stripe mode test ou simulation) et paiement à la livraison.
- Suivi en temps réel sur sept statuts, avec historique horodaté.
- Notation du restaurant après livraison, avec recalcul de la note moyenne.
- Espaces restaurant, livreur et administrateur.
- Discussion rattachée à chaque commande.
- Carte de livraison affichant la position réelle du livreur.
- Gestion du menu par le restaurant : plats, options et suppléments.
- Création d'un établissement et de son gestionnaire par l'administration.
- Notifications à chaque étape de la commande.
- Configuration de déploiement versionnée pour Render et Railway.

Exclus

- Connexion Google et authentification à deux facteurs.
- Pourboires et modération des avis.
- Notifications distantes démontrables sans development build.
- Auto-suppression du compte par l'utilisateur.

3. Personas et parcours utilisateur

Persona 1 — Maya, cliente

24 ans, étudiante à Montréal. Commande deux à trois fois par semaine, souvent entre deux cours. Ce qui compte pour elle : savoir en combien de temps son repas arrive, pouvoir retirer les oignons de son burger, et payer sans sortir sa carte à chaque fois.

Persona 2 — Karim, gestionnaire de restaurant

38 ans, gère une pizzeria. En plein coup de feu, il n'a pas le temps de rafraîchir un écran. Il veut voir la commande arriver toute seule et changer son statut en un seul geste.

Persona 3 — Luc, livreur

29 ans, livre à vélo. Il veut voir les courses disponibles autour de lui et être certain qu'une course acceptée est bien la sienne — pas de mauvaise surprise en arrivant au restaurant.

Persona 4 — Sarah, administratrice

31 ans, exploite la plateforme. Elle a besoin d'un état de l'activité et de pouvoir intervenir : désactiver un restaurant fermé, annuler une commande bloquée, corriger le rôle d'un compte.

Parcours complet — de la faim à la note

1. Maya ouvre l'application. Sa session est restaurée : elle arrive directement sur le catalogue.
2. Elle cherche « burger », ouvre la fiche d'Atelier Burger, choisit Le Signature et retire les oignons. Le prix se met à jour.
3. Elle passe au panier, puis au paiement. Le serveur recalcule les taxes du Québec et le total.
4. Elle paie par carte. La commande n'est enregistrée qu'une fois le paiement accepté.
5. Chez Karim, la commande apparaît sans qu'il touche à rien. Il la confirme, la prépare, la marque prête.
6. La course apparaît chez tous les livreurs. Luc l'accepte : elle disparaît de l'écran des autres.
7. Luc passe « en route ». Maya voit le changement en direct et peut lui écrire pour préciser le code de l'immeuble.
8. Luc livre. Le bouton « Noter le restaurant » apparaît chez Maya.
9. Maya met 5 étoiles. La note d'Atelier Burger est recalculée et visible sur sa fiche.
10. Sarah voit la commande apparaître dans les statistiques de la plateforme.

4. Fonctionnalités livrées — backlog produit

ID	User story	Priorité	Critère d'acceptation déterminant	État
US-01	Parcourir et personnaliser les plats	Must	Le prix se met à jour en direct ; une option inconnue est ignorée par le serveur	Livré
US-02	Suivre ma commande en temps réel	Must	Le statut change en moins de 5 secondes, sans action de l'utilisateur	Livré
US-03	Payer ma commande en ligne	Must	Aucune commande n'est créée si le paiement échoue	Livré
US-04	Noter le restaurant après livraison	Must	Notation possible après livraison seulement, et une seule fois	Livré
US-05	Recevoir les commandes (restaurant)	Must	Cloisonnement : la commande d'un autre restaurant renvoie 403	Livré

ID	User story	Priorité	Critère d'acceptation déterminant	État
US-06	Accepter une livraison (livreur)	Should	Deux livreurs simultanés : un seul obtient la course	Livré
US-07	Discuter au sujet d'une commande	Could	Seules les parties prenantes accèdent à la discussion	Livré
US-08	Superviser la plateforme (admin)	Could	Toute route d'administration renvoie 403 à un non-administrateur	Livré
US-09	Retrouver ma session à la réouverture	Should	Le jeton est conservé dans le trousseau sécurisé du système	Livré
US-10	Gérer mon menu (restaurant)	Should	Un gestionnaire ne modifie que les plats de son établissement ; sinon 403	Livré
US-11	Créer un restaurant (admin)	Should	L'établissement et son gestionnaire sont créés en une opération	Livré
US-12	Être averti à chaque étape (client)	Could	Une notification s'affiche à chaque changement de statut	Livré

5. Architecture technique

5.1 Vue d'ensemble

L'application mobile dialogue avec le backend par deux canaux complémentaires : une API REST pour les opérations classiques (s'inscrire, commander, payer) et un canal Socket.IO pour tout ce qui doit apparaître sans rafraîchissement (nouvelles commandes, changements de statut, messages).

Le canal temps réel n'est pas une porte dérobée : un socket sans jeton JWT valide est refusé à la poignée de main, et les autorisations y sont les mêmes que sur l'API.

Figure 1 — Architecture client-serveur : docs/diagrammes/architecture.svg

5.2 Découpage du backend

Dossier	Responsabilité
config/	Variables d'environnement validées au démarrage, connexion MongoDB, salons Socket.IO
middleware/	Authentification, autorisation par rôle, sécurité HTTP, gestion des erreurs
services/	Règles métier pures : tarification, statuts, validation du menu, paiement, notifications — testables sans base de données
models/	Schémas Mongoose et index

Dossier	Responsabilité
controllers/	Orchestration HTTP : validation, appel des services, réponse
routes/	Déclaration des points d'entrée et des protections
utils/	Enveloppe des contrôleurs asynchrones, échappement des expressions régulières

Pourquoi un dossier services/

Les règles métier étaient auparavant mélangées au code HTTP des contrôleurs. Le taux de taxes existait à deux endroits, la machine à états à un seul mais impossible à tester sans démarrer le serveur. Les extraire a permis d'écrire 30 tests unitaires qui s'exécutent en moins d'une seconde, sans MongoDB ni réseau.

5.3 Modèle de données

Sept collections MongoDB : utilisateurs, restaurants, plats, commandes, messages, avis, et les appareils enregistrés pour les notifications.

Décision de modélisation	Raison
Le nom et le prix des plats sont recopiés dans la commande	Si le restaurant change son menu, la commande conserve ce qui a été réellement payé. Une commande est une pièce comptable, pas une vue sur le menu
historiqueStatuts conserve chaque changement horodaté	Permet d'estimer le temps restant et de justifier a posteriori le déroulement d'une livraison
Index unique sur avis.commandeId	Une commande ne peut être notée qu'une fois. La contrainte est portée par la base : deux requêtes simultanées ne peuvent pas créer deux avis
Index composé { utilisateurId, createdAt: -1 }	L'historique d'un client est la requête la plus fréquente de l'application
restaurants.note n'est jamais saisie	Elle est recalculée par agrégation à partir des avis réels, ce qui la rend juste même en cas de notations simultanées

Figure 2 — Collections MongoDB et relations : docs/diagrammes/schema_mongodb.svg

5.4 Machine à états d'une commande

Huit statuts et des transitions strictement contrôlées. Toute transition non prévue est refusée avec un code 409, quel que soit le rôle de l'appelant.

Statut	Transitions autorisées	Qui peut la déclencher
en attente	confirmée, annulée	Restaurant
confirmée	en préparation, annulée	Restaurant
en préparation	prête, annulée	Restaurant
prête	prise en charge	Livreur (acceptation)
prise en charge	en route	Livreur
en route	livrée	Livreur
livrée	aucune — état terminal	—
annulée	aucune — état terminal	—

Une commande ne peut plus être annulée une fois « prête » : le repas est déjà préparé. C'est une règle métier, pas une limitation technique.

5.5 Attribution des livraisons

L'attribution est volontaire, pas automatique : aucun algorithme n'affecte de livreur. C'est un modèle « pull », où le livreur choisit sa course.

Une commande n'est jamais proposée aux livreurs au moment où elle est passée. Elle ne le devient qu'au statut « prête », posé par le restaurant — inutile d'envoyer quelqu'un chercher un repas qui n'est pas prêt.

Comment deux livreurs simultanés sont départagés

Le filtre { statut: prête, livreurId: null } accompagne la mise à jour. MongoDB garantit qu'une modification de document est atomique : le premier livreur passe, le second ne correspond plus au filtre et reçoit un 409. Il n'y a ni verrou ni transaction, et c'est précisément ce qui rend la chose correcte — un « lire puis écrire » aurait laissé une fenêtre où les deux livreurs s'écrasent mutuellement.

Limites assumées de ce modèle :

Limite	Conséquence	Piste d'évolution
Aucune notion de proximité	Un livreur à l'autre bout de la ville voit la même course que celui devant le restaurant	Filtrage par distance
Aucune réattribution	Si personne n'accepte, la commande reste disponible indéfiniment	Délai d'expiration puis escalade

Limite	Conséquence	Piste d'évolution
Pas de désistement	Seul un administrateur peut annuler après acceptation	Route remettant livreurId à null

5.6 Organisation du temps réel

Salon Socket.IO	Qui le rejoint	Ce qu'il reçoit
utilisateur:<id>	Tout compte connecté	Ses propres commandes
role:livreur	Les livreurs	Les courses disponibles
role:admin	Les administrateurs	Toute l'activité de la plateforme
restaurant:<id>	Les comptes restaurant	Les commandes de leur établissement
commande:<id>	À la demande	Les messages de la discussion

5.7 Gestion du catalogue

Le catalogue n'est plus figé dans un script de démarrage. Deux rôles le font évoluer, à travers un contrôleur unique.

Rôle	Portée	Routes
Administrateur	N'importe quel établissement	/api/admin/restaurants et /api/admin/plats
Gestionnaire de restaurant	Uniquement le sien	/api/mon-restaurant

Le cloisonnement ne repose pas sur la route appelée mais sur une fonction de résolution qui vérifie le droit d'accès à chaque appel. Un gestionnaire qui viserait un autre établissement par l'URL reçoit un 403 explicite, et non une redirection silencieuse.

Deux règles à souligner en présentation

La note d'un restaurant n'est jamais acceptée en écriture : elle est calculée à partir des avis réels, sans quoi un formulaire permettrait de la truquer. Et retirer un plat ne le supprime pas : il devient indisponible, car des commandes passées le référencent et l'effacer casserait l'historique.

Le script de chargement des données de démonstration est devenu non destructif : il crée ce qui manque et met à jour ce qui existe. Les restaurants créés depuis l'application survivent donc à un rechargement.

5.8 Notifications

Deux canaux complémentaires, pour la même raison que le double mode de paiement : la démonstration doit fonctionner dans Expo Go.

Canal	Mécanisme	Portée	Limite
Push distantes	Le serveur appelle le service Expo Push	Atteint l'appareil application fermée	Ne fonctionne plus dans Expo Go sur Android depuis le SDK 53
Notification locale	L'application l'affiche à la réception d'un événement Socket.IO	Fonctionne partout, Expo Go compris	Uniquement application ouverte

Les erreurs d'envoi sont absorbées et journalisées : une notification perdue est moins grave qu'un changement de statut qui échouerait à cause d'elle. Les jetons signalés comme désinscrits par Expo sont supprimés automatiquement, et le jeton est retiré à la déconnexion pour qu'un téléphone partagé ne reçoive pas les commandes du compte précédent.

5.9 Carte de livraison : deux implémentations

La carte utilise react-native-maps sur mobile et Leaflet avec OpenStreetMap sur le web. La séparation a demandé deux précautions.

Piège rencontré	Pourquoi cela échoue	Solution retenue
Import conditionnel selon la plateforme	Metro analyse les require() à la compilation : la condition d'exécution arrive trop tard, le module natif entre quand même dans le paquet	Deux fichiers distincts
Fichier .web.tsx placé dans le dossier des routes	Expo Router énumère tous les fichiers de src/app avec require.context : les deux variantes entrent dans le graphe	Composants déplacés dans src/components/, la route se contente de réexporter

Leaflet a été préféré à Google Maps : aucune clé d'API, aucun compte de facturation, aucune dépendance npm supplémentaire. En contrepartie, la carte web dépend d'un CDN et ne suit pas la position du livreur en continu, contrairement à la version mobile.

5.10 Séquence — passer une commande

Figure 3 — Du panier au paiement, puis suivi et notation : docs/diagrammes/sequence_commande.svg

6. Fiche de cas d'utilisation — Passer une commande

Élément	Description
Identifiant	UC-01
Acteur principal	Client authentifié
Acteurs secondaires	Service de paiement, restaurant, livreur
Préconditions	Le client est connecté ; le panier contient au moins un plat d'un restaurant actif
Postconditions	Une commande existe au statut « en attente », le paiement est réglé ou différé, le restaurant est notifié
Scénario nominal	1. Le client valide son panier. 2. Il saisit son adresse et choisit un mode de paiement. 3. Le serveur relit les prix en base et calcule le total. 4. Le paiement est traité. 5. La commande est enregistrée. 6. Elle est diffusée au restaurant par Socket.IO.
Alternative A — paiement à la livraison	L'étape 4 est ignorée ; le statut de paiement est « à payer » et devient « payé » à la livraison
Exception E1 — plat indisponible	Le serveur renvoie 400 ; aucune commande n'est créée ; le panier est conservé
Exception E2 — paiement refusé	Le serveur renvoie 402 avec le motif ; aucune commande n'est créée ; le panier est conservé
Exception E3 — restaurant désactivé	Le serveur renvoie 404 ; le client est invité à choisir un autre restaurant
Règle de sécurité	Les montants envoyés par le client sont ignorés : les prix sont toujours relus en base

7. Sécurité et conformité

7.1 Mesures mises en œuvre

Menace	Mesure	Où
Vol de mots de passe	Hachage bcrypt à 12 tours ; le champ n'est jamais renvoyé par l'API	models/ Utilisateur.js
Force brute sur la connexion	10 tentatives par 15 minutes et par adresse IP ; réponse 429	middleware/ securite.js
Élévation de privilèges	Le rôle ne peut être ni choisi à l'inscription, ni modifié par l'utilisateur	controllers/ authController.js
Jeton d'un compte révoqué	L'utilisateur est rechargé en base à chaque requête : un compte supprimé perd ses droits immédiatement	middleware/auth.js

Menace	Mesure	Où
Accès croisé entre comptes	Vérification de propriété dans chaque contrôleur, jamais par simple masquage d'un bouton	controllers/
Déni de service par regex	Les saisies de recherche sont échappées avant d'entrer dans un \$regex	utils/texte.js
Falsification du montant	Les prix sont relus en base ; les montants du client sont ignorés	services/ tarification.js
Exposition des données de carte	Aucun numéro stocké ; seule la référence de transaction est conservée	services/ paiement.js
Requêtes d'origine inconnue	Politique CORS par liste blanche, obligatoire en production	middleware/ securite.js
Vol du jeton sur l'appareil	Jeton conservé dans le trousseau système, pas dans un stockage en clair	services/stockage.ts
Note de restaurant truquée	La note n'est jamais acceptée en écriture ; elle est calculée depuis les avis	services/ validationMenu.js
Menu d'un autre restaurant modifié	Résolution du restaurant vérifiée à chaque appel, jamais déduite de l'URL	controllers/ menuController.js
Notifications d'un compte précédent	Le jeton d'appareil est retiré à la déconnexion	services/ notifications.ts

7.2 Conformité Loi 25

- Consentement recueilli à l'inscription ; autorisation explicite du système pour la géolocalisation.
- Minimisation : seules les données nécessaires au service sont collectées.
- Sécurité : hachage, jetons à durée limitée, en-têtes de sécurité HTTP.
- Exactitude : l'utilisateur peut corriger son profil à tout moment.
- Droit à l'effacement : suppression disponible côté administration ; l'auto-suppression reste à implémenter.

Garde-fou volontaire

Le serveur refuse de démarrer si le secret JWT fait moins de 32 caractères, si l'URI MongoDB manque, ou si une clé Stripe de production est fournie. Un message clair au démarrage vaut mieux qu'une faille silencieuse.

8. Qualité : tests et intégration continue

Niveau	Outil	Couverture	Automatisé
Unitaire	node:test	Tarification, statuts, sécurité, erreurs asynchrones, validation du menu, notifications — 30 tests	Oui
Statique	tsc --noEmit	Cohérence des types entre l'API et les écrans mobiles	Oui
Intégration API	Postman / curl	Enchaînement des routes, codes de statut, autorisations	Non
Acceptation	Parcours sur téléphone	Scénario complet multi-rôles et cas d'erreur	Non

L'intégration continue GitHub Actions exécute les tests du backend et la vérification des types du mobile à chaque Pull Request sur main et develop.

Cas d'erreur vérifiés

Situation testée	Résultat attendu
Deux livreurs acceptent la même course	Un seul l'obtient ; l'autre reçoit 409
Un restaurant consulte la commande d'un autre établissement	403
Noter deux fois la même commande	409 (index unique en base)
Noter une commande non livrée	409
Carte de test refusée par Stripe	402, panier conservé
11 tentatives de connexion en 15 minutes	429 avec délai de réessai
Recherche contenant « (a+)+\$ »	Réponse immédiate, aucun blocage du serveur
Un gestionnaire modifie le plat d'un autre restaurant	403
Une note de 5 est imposée à la création d'un restaurant	Ignorée : le restaurant est créé avec une note de 0
Un jeton de notification hors format Expo est envoyé	400

9. Gestion de projet

9.1 Organisation Scrum

Membre	Rôle technique	Chapeau Scrum
JORDAN DONGMEZA	Lead technique / Backend	Scrum Master
YAN SAH	Frontend React Native	Développement

Membre	Rôle technique	Chapeau Scrum
WILFRED GALIMAR	Intégrations et temps réel	Développement
ARCHANGE GUIMDO	QA / DevOps / Coordination documentaire	Product Owner

Trois sprints de quatre semaines, avec une revue à mi-sprint pour compenser une boucle de retour plus longue qu'avec des sprints courts.

9.2 Conventions Git

- Aucun push direct sur main ; le travail part de develop.
- Une branche par tâche : feature/MEAL-18-auth-backend.
- Commits normés : feat, fix, docs, test, refactor, chore.
- Une Pull Request par tâche, revue par un coéquipier.

9.3 Documentation as code

La documentation vit dans le dépôt, en Markdown, et se met à jour dans la même Pull Request que le code qu'elle décrit. Une tâche n'est terminée que si le code, les tests et la documentation sont à jour.

Cette règle a été prise au sérieux tardivement : à la semaine 11, le schéma de base de données et la documentation d'API décrivaient encore l'état du Sprint 1. La reprise a été menée en une passe complète, et cinq ADR ont été rédigés — dont trois rétroactivement — pour retracer les décisions prises en cours de route.

10. Difficultés rencontrées et solutions

10.1 Le suivi temps réel n'était pas démontrable

Problème — Les statuts « en route » et « livrée » n'avaient aucun acteur pour les déclencher, l'application livreur étant hors périmètre. La fonctionnalité 2, imposée par le cahier des charges, n'aurait pu être présentée qu'en modifiant la base à la main.

Solution — Élargissement à quatre rôles dans une seule base de code, avec redirection selon le rôle après connexion. Documenté dans l'ADR 0005.

10.2 Le SDK Stripe ne fonctionne pas dans Expo Go

Problème — Le composant de saisie de carte de Stripe exige une compilation native, incompatible avec l'outil utilisé pour les démonstrations sur téléphone. Par ailleurs, une démonstration dépendante du réseau était un risque pour l'évaluation.

Solution — Un service unique choisit automatiquement entre Stripe en mode test et une simulation locale, selon la présence d'une clé. Le numéro saisi ne sert qu'à sélectionner un moyen de paiement de test : le serveur ne manipule aucune donnée de carte réelle. Documenté dans l'ADR 0004.

10.3 Les erreurs asynchrones du backend étaient silencieuses

Problème — Une promesse rejetée dans un contrôleur n'atteignait jamais le gestionnaire d'erreurs d'Express. La requête restait en attente jusqu'au délai d'expiration et le client affichait « le serveur ne répond pas », alors que le vrai problème était tout autre. C'est le défaut le plus grave corrigé sur le projet.

Solution — Une enveloppe asynchrone appliquée à toutes les routes transmet le rejet au middleware d'erreurs, qui renvoie un code et un message exploitables. Un test unitaire garantit la non-régression.

10.4 Adresses IP écrites en dur

Problème — L'adresse du serveur apparaissait dans trois fichiers différents. L'application cessait de fonctionner à chaque changement de réseau, et un fichier de documentation portait même une adresse dans son nom.

Solution — Toute la configuration réseau passe par un fichier unique alimenté par le .env d'Expo. Au démarrage, le serveur affiche les adresses réellement détectées sur la machine.

10.5 Session perdue à chaque fermeture

Problème — Le jeton ne vivait qu'en mémoire React : fermer l'application déconnectait l'utilisateur.

Solution — Le jeton est conservé dans le trousseau sécurisé du système. Au lancement, la session est restaurée puis validée auprès du serveur, ce qui récupère au passage le rôle à jour.

10.6 Aucun moyen d'ajouter un restaurant

Problème — Le catalogue n'existait que dans le script de démarrage, qui effaçait tout avant de recharger. L'administration pouvait lister, désactiver et supprimer un établissement, mais pas en créer. Ajouter un quatrième restaurant imposait d'éditer le script et de perdre les trois autres, ainsi que toutes les notes accumulées.

Solution — Un contrôleur unique de gestion des menus, partagé par l'administration et les gestionnaires de restaurant, avec vérification du droit d'accès à chaque appel. La création d'un établissement crée aussi, en une opération, le compte de son gestionnaire. Le script de chargement est devenu non destructif.

10.7 Notifications impossibles dans Expo Go

Problème — Depuis le SDK 53, Expo Go ne reçoit plus les notifications distantes sur Android. Toutes les démonstrations se font dans Expo Go : implémenter uniquement les push revenait à livrer une

fonctionnalité invisible le jour de la présentation. C'est exactement le mur déjà rencontré avec le SDK Stripe.

Solution — Deux canaux envoyés systématiquement : les vraies push distantes côté serveur, et une notification locale déclenchée par l'événement Socket.IO côté application. Les deux ne font pas doublon, puisqu'Expo Go ne reçoit jamais les push distantes. Documenté dans l'ADR 0006.

10.8 Fonctionnalité imposée oubliée

Problème — La notation des restaurants, quatrième fonctionnalité imposée, n'existait ni côté serveur ni côté mobile en semaine 11. Le temps consacré aux espaces restaurant, livreur et administration l'avait repoussée.

Solution — Implémentation complète : modèle avec index unique, routes de dépôt et de consultation, recalcul de la note par agrégation, écran de notation et point d'entrée depuis le suivi et l'historique.

11. Bilan par rapport au cahier des charges

Engagement	État	Commentaire
Menu interactif et personnalisation	Atteint	Options avec supplément, prix mis à jour en direct
Suivi de commande en temps réel	Atteint	Sept statuts, propagation constatée sous la seconde en réseau local
Paieement intégré	Atteint	Stripe en mode test, avec repli hors ligne
Notation des restaurants	Atteint	Livrée en semaine 11 après un report interne
Documentation complète et à jour	Atteint	Reprise intégrale de /docs et cinq ADR
Historique Git lisible	Atteint	Un commit par fonctionnalité, branches main et develop
Tests automatisés	Partiel	30 tests unitaires ; aucun test d'intégration automatisé
50 utilisateurs simultanés	Non vérifié	Aucun test de charge réalisé
Backend déployable	Atteint	Configurations Render et Railway versionnées, durcissement production appliqué
Gestion du catalogue	Atteint	Création et édition depuis l'application, par l'admin et le restaurant
Notifications à chaque étape	Atteint	Deux canaux ; les push distantes exigent un developement build

12. Dette technique et perspectives

Élément	Conséquence	Correction envisagée
Pas de test d'intégration automatisé	Les régressions d'autorisation ne sont vues qu'en test manuel	Supertest + MongoDB en mémoire
Pas de test de charge	Une exigence chiffrée reste invérifiée	Campagne k6 ou Artillery
Limitation de débit en mémoire	Inefficace derrière plusieurs instances	Compteur partagé via Redis
Pas d'envoi de courriel	Le jeton de réinitialisation transite par la réponse en développement	Service transactionnel
Positions fixes sur la carte	Le trajet affiché n'est pas fidèle	Géocodage et diffusion continue de la position
Attribution sans proximité ni réattribution	Une course non acceptée reste disponible sans limite de temps	Filtrage par distance et délai d'expiration
Notifications distantes non démontrables	Seul le canal local est visible en présentation	Development build EAS
Libellés de notification dupliqués	Serveur et application peuvent diverger sans alerte	Source unique exposée par l'API

Perspectives fonctionnelles

- Pourboires et modération des avis.
- Connexion Google et authentification à deux facteurs pour l'administration.
- Auto-suppression du compte par l'utilisateur, exigée par le droit à l'effacement.

13. Scénario de démonstration

Déroulé recommandé, à trois appareils si possible. Durée visée : 12 minutes.

#	Ce qu'on montre	Ce qu'il faut souligner	Durée
1	Ouverture de l'application, session déjà restaurée	Le jeton est conservé dans le trousseau du système	30 s
2	Recherche, fiche restaurant, personnalisation d'un plat	Le prix se met à jour ; le serveur revalide les options	1 min 30
3	Panier et paiement par carte de test	Les taxes viennent du serveur, pas du téléphone	1 min 30
4	Écran restaurant : la commande apparaît seule	Aucun rafraîchissement, aucune action	1 min

#	Ce qu'on montre	Ce qu'il faut souligner	Durée
5	Progression jusqu'à « prête »	Les transitions interdites sont refusées	1 min
6	Écran livreur : acceptation de la course	Attribution atomique — le montrer sur deux appareils	1 min 30
7	Discussion client / livreur	Même canal temps réel, mêmes autorisations	1 min
8	Livraison puis notation	Une seule note par commande ; la moyenne est recalculée	1 min 30
9	Tableau de bord administrateur, création d'un restaurant	L'établissement et son gestionnaire sont créés en une opération	1 min
9 bis	Le gestionnaire ajoute un plat avec options	Il apparaît aussitôt côté client	1 min
10	Un cas d'erreur au choix (403, 409 ou 402)	La sécurité est appliquée côté serveur, pas par masquage d'un bouton	1 min

Avant la démonstration

Vérifier dans l'ordre : MongoDB démarré, npm run seed exécuté, backend lancé, /api/sante accessible depuis le navigateur du téléphone, .env mobile à jour avec l'adresse affichée par le backend, Expo relancé avec --clear, autorisation des notifications acceptée. Les trois comptes de démonstration doivent exister. Si le backend est déployé sur l'offre gratuite de Render, ouvrir /api/sante cinq minutes avant : la première requête après inactivité met environ 50 secondes.

À dire pendant la démonstration des notifications

Ce qui s'affiche dans Expo Go est la notification locale. Les notifications distantes sont implémentées et fonctionnelles, mais Expo Go ne les reçoit plus sur Android depuis le SDK 53 : elles demandent un development build. Énoncer ce point est plus solide que de laisser croire à une push distante.

14. Annexes

Annexe A — Arborescence du projet

Chemin	Contenu
backend/src/	API REST, Socket.IO, services métier, modèles
backend/tests/	30 tests unitaires exécutés par node --test
mobile/App-Client/src/app/	Écrans, routés par fichiers via expo-router
mobile/App-Client/src/components/	Composants partagés, dont les deux implémentations de la carte

Chemin	Contenu
mobile/App-Client/src/context/	Session et panier
mobile/App-Client/src/services/	Client API typé et stockage sécurisé du jeton
docs/	Documentation technique vivante (architecture, base de données, API, tests, ADR)
livrables/	Documents formels remis au cours
.github/workflows/	Intégration continue

Annexe B — Commandes utiles

Commande	Effet
cd backend && npm run dev	Démarre l'API et Socket.IO, affiche les adresses réseau
cd backend && npm run seed	Recharge le catalogue de démonstration
cd backend && npm test	Exécute les 30 tests unitaires
cd backend && npm run seed:reinitialiser	Repart d'un catalogue vide (destructif)
cd backend && npm run creer-admin	Crée un compte administrateur
cd mobile/App-Client && npx expo start --clear --lan	Démarre l'application mobile
cd mobile/App-Client && npm run typecheck	Vérifie les types de toute l'application

Annexe C — Cartes de test

Numéro	Résultat
4242 4242 4242 4242	Païement accepté (Visa)
5555 5555 5555 4444	Païement accepté (Mastercard)
4000 0000 0000 0002	Païement refusé — utile pour démontrer le chemin d'erreur

Expiration : une date future, par exemple 12/30. CVV : trois chiffres quelconques. Titulaire : au moins trois caractères.

Annexe D — Décisions d'architecture

ADR	Sujet	Décision
0001	Choix de React Native	Une seule base de code pour Android et iOS
0002	Choix de MongoDB	Modèle documentaire adapté aux commandes imbriquées

ADR	Sujet	Décision
0003	TypeScript et Expo Router	Types vérifiés à la compilation, routage par fichiers
0004	Paielement	Stripe en mode test avec repli sur simulation
0005	Multi-rôles	Quatre espaces pour rendre le suivi temps réel démontrable
0006	Notifications	Push distantes avec repli local, pour rester démontrable dans Expo Go